

CSE 341 Part 1 Design Specification

BudgetLang

Student ID: 210104004817

May 2026

4.1 Language Overview [P1]

BudgetLang is a domain-specific language for budgeting and personal finance tasks. Its main purpose is to let users describe transactions, spending limits, and simple budget-oriented control flow using a small set of clear constructs instead of a full general-purpose language. The design targets budgeting scenarios such as recording income and expenses, grouping transactions into arrays, computing totals, and attaching simple reporting commands such as **budget** and **track**.

BudgetLang prioritizes readability, writability, and reliability in the sense discussed by Sebesta (§1.3). Readability is improved by a small number of statement forms, explicit type names, and domain-specific commands whose meaning is visible from their syntax. Writability is supported through user-defined functions, arrays of transactions, and concise commands such as **budget** and **track**. Reliability is supported by the use of explicit declarations and a restricted syntax instead of many competing forms. The main trade-off is cost and generality (§1.3.4, §1.6): BudgetLang is intentionally less flexible than a general-purpose language because it focuses on budgeting tasks rather than arbitrary programming problems.

Sample Program

```
money totalExpenses(transaction[] items) {
    money total = $0;

    foreach (transaction t in items) {
        if (t.kind == "expense") {
            total = total + t.amount;
        }
    }

    return total;
}

money monthlyIncome = $3000;
money rentLimit = $1200;

transaction rent;

rent.amount = $1100;
rent.category = "housing";
rent.kind = "expense";
rent.note = "May rent";

transaction salary;
```

```

salary.amount = $3000;
salary.category = "job";
salary.kind = "income";
salary.note = "Monthly salary";

budget "Rent" rentLimit;
track rent;
track salary;

transaction[] records = [rent, salary];

money spent = totalExpenses(records);

if (spent > rentLimit) {
    transaction mylife;

    mylife.amount = $50;
    mylife.category = "warning";
    mylife.kind = "expense";
    mylife.note = "Over budget";

    track mylife;
} else {
    budget "Status" spent;
}

int month = 5;

switch (month) {
    case 5:
        budget "Month" monthlyIncome;
    default:
        track salary;
}

```

4.2 Lexical Structure [P1]

BudgetLang separates lexical analysis from syntax analysis, following Sebesta's discussion of compiler front ends (§4.1-§4.2). The lexer groups input characters into lexemes and returns token categories to the parser.

Whitespace is ignored except as a separator between lexemes and for line counting in error reporting. The special words of BudgetLang are treated as reserved words rather than ordinary identifiers, which improves clarity and avoids ambiguity (§5.2.3).

4.3 Syntax [P1]

The syntax of BudgetLang is described below in EBNF, following Sebesta's discussion of context-free grammars and EBNF (§3.3.1-§3.3.2). The grammar is intended to be unambiguous for the implemented Part 1 subset.

Ambiguity Resolution Notes

- Operator precedence is encoded directly in the grammar rather than being left to informal convention (§3.3.1.8-§3.3.1.9).

Category	Pattern	Examples
Identifiers	<code>letter (letter digit _)*</code>	<code>totalExpenses, rent, salary2, records</code>
Keywords	Reserved words	<code>money, int, string, transaction, if, else, foreach, in, return, budget, track, switch, case, default</code>
Money literals	<code>"\$" digit+</code>	<code>\$0, \$50, \$1200, \$3000</code>
String literals	Text enclosed in double quotes	<code>"Rent", "expense", "May rent", "Status"</code>
Int literals	<code>digit+</code>	<code>5, 0, 1, 9999</code>
Operators	Fixed operator symbols	<code>=, +, -, *, /, ., ==, !=, <, <=, >, >=</code>
Separators	Fixed separator symbols	<code>() { } [] ; , :</code>

- The precedence order is: parentheses, field access, multiplicative operators, additive operators, relational operators, then equality operators.
- Binary operators at the same level are left-associative because they are written using repeated EBNF forms.
- The optional `else` is attached to the nearest unmatched `if`.
- Assignment is treated as a statement, not as an expression.

EBNF Grammar

`<program> ::= { <function_declaration> | <statement> }`

`<function_declaration> ::= <type> <identifier> "(" [ <parameter_list> ] ")" <block>`

`<parameter_list> ::= <parameter> { ",", <parameter> }`

`<parameter> ::= <type> <identifier>`

`<type> ::= "money"
| "int"
| "string"
| "transaction"
| "transaction" "[" "]"`

`<block> ::= "{" { <statement> } "}"`

`<statement> ::= <var_declaration>
| <assignment_stmt>
| <field_assignment_stmt>
| <if_stmt>
| <foreach_stmt>
| <switch_stmt>
| <return_stmt>
| <budget_stmt>
| <track_stmt>`

```

    | <call_stmt>

<var_declaration> ::= <type> <identifier> [ "=" <expression> ] ";"

<assignment_stmt> ::= <identifier> "=" <expression> ";"

<field_assignment_stmt> ::= <identifier> "." <identifier> "=" <expression> ";"

<if_stmt> ::= "if" "(" <expression> ")" <block>
    | "if" "(" <expression> ")" <block> "else" <block>

<foreach_stmt> ::= "foreach" "(" <type> <identifier> "in" <expression> ")" <block>

<switch_stmt> ::= "switch" "(" <expression> ")" "{"
    { <case_clause> }
    [ <default_clause> ]
    "}"

<case_clause> ::= "case" <case_value> ":" { <statement> }

<default_clause> ::= "default" ":" { <statement> }

<case_value> ::= <int_literal> | <string_literal>

<return_stmt> ::= "return" <expression> ";"

<budget_stmt> ::= "budget" <string_literal> <expression> ";"

<track_stmt> ::= "track" <expression> ";"

<call_stmt> ::= <function_call> ";"

<expression> ::= <equality_expr>

<equality_expr> ::= <relational_expr> { ( "==" | "!=" ) <relational_expr> }

<relational_expr> ::= <additive_expr> { ( "<" | "<=" | ">" | ">=" ) <additive_expr> }

<additive_expr> ::= <multiplicative_expr> { ( "+" | "-" ) <multiplicative_expr> }

<multiplicative_expr> ::= <postfix_expr> { ( "*" | "/" ) <postfix_expr> }

<postfix_expr> ::= <primary_expr> { "." <identifier> }

<primary_expr> ::= <money_literal>
    | <int_literal>
    | <string_literal>
    | <identifier_expr>
    | <array_literal>
    | "(" <expression> ")"

```

```

<identifier_expr> ::= <identifier> [ "(" [ <argument_list> ] ")" ]

<function_call> ::= <identifier> "(" [ <argument_list> ] ")"

<argument_list> ::= <expression> { "," <expression> }

<array_literal> ::= "[" [ <argument_list> ] "]"

<identifier> ::= letter { letter | digit | "_" }

<money_literal> ::= "$" digit { digit }

<int_literal> ::= digit { digit }

<string_literal> ::= "'" { <string_character> } "'"

<string_character> ::= letter | digit | " " | "_"

```

4.6 Names, Binding, Scope, Lifetime [P1]

Legal Identifiers

Identifiers in BudgetLang begin with a letter and may continue with letters, digits, or under-scores. This rule supports readability by preventing confusing names that begin with digits. Reserved words such as `money`, `if`, and `budget` cannot be reused as programmer-defined names (§5.2.3).

Compile-Time and Run-Time Bindings

Several bindings are determined before execution. The set of reserved words, the meaning of punctuation symbols, the declared type of each variable, the declared return type of each function, and the names of record fields are all statically bound. This design follows Sebesta's discussion of static binding and static type binding (§5.4.1-§5.4.2). At run time, variables become bound to particular values, function parameters become bound to argument values when calls occur, and transaction fields receive values through assignment statements.

Scope Rule

BudgetLang uses static scope rather than dynamic scope (§5.5.1). Under static scope, a name is resolved by the textual block structure of the program, which improves readability and makes the referencing environment easier to determine from the source code. Each function body introduces a new local scope, and nested blocks introduced by `if`, `foreach`, and `switch` may contain local declarations. If BudgetLang were changed to dynamic scope, a function such as `totalExpenses` could depend on names from its calling environment rather than from its own declarations and parameters, which would make reasoning about the program much harder.

Lifetime of Local Variables

The lifetime of local variables is stack-dynamic (§5.4.3, §5.6). Local variables and parameters are created when control enters their block or function activation and cease to exist when that activation finishes. This choice fits recursive function calls and ordinary block-structured execution. Function declarations themselves have program-wide visibility after parsing, but their activation records and locals are created only when the functions are called.